

**UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS**

Plan de Gestión del Alcance

MalphasOS
Gestión de Clientes

Sean Sebastián Bolívar Calderón - 20231020135

2 de septiembre de 2026

Contents

Identificación del plan	2
Nota preliminar	2
Qué describe este plan	2
El responsable es uno solo	2
Los artefactos que sostienen la gestión del alcance	2
1. Proceso de definición del alcance	4
1.1. Qué	4
1.2. Quién	4
1.3. Cómo	4
1.4. Cuándo	5
1.5. Dónde	5
1.6. Con qué	6
2. Proceso de elaboración de la EDT/WBS	7
2.1. Qué	7
2.2. Quién	7
2.3. Cómo	7
2.4. Cuándo	8
2.5. Dónde	8
2.6. Con qué	9
3. Proceso para la verificación del alcance	10
3.1. Qué	10
3.2. Quién	10
3.3. Cómo	10
3.4. Cuándo	12
3.5. Dónde	12
3.6. Con qué	12
4. Proceso para el control del alcance	13
4.1. Qué	13
4.2. Quién	13
4.3. Cómo	14
4.4. Cuándo	15
4.5. Dónde	15
4.6. Con qué	16

Identificación del plan

Nombre del plan de gestión del alcance: Plan de Gestión del Alcance de MalphasOS

Nombre del proyecto: MalphasOS

Siglas del proyecto: MSO

Nota preliminar

Qué describe este plan

Este documento no es una plantilla rellena. Describe el proceso de gestión del alcance que MalphasOS **sigue de hecho**, tal como puede comprobarse en su repositorio: el historial de *git*, las migraciones de base de datos, la batería de pruebas, el wiki técnico y la propia Especificación de Requisitos de Software (ERS). Cada afirmación sobre el proceso remite a un artefacto concreto que el lector puede abrir. Cuando el proyecto no tiene nada que poner en un campo, el documento lo dice en lugar de inventarlo.

Las cifras que aparecen a lo largo del texto están tomadas del estado del proyecto al **2 de septiembre de 2026**.

El responsable es uno solo

Todos los roles de este plan recaen en la misma persona: Sean Sebastián Bolívar Calderón desempeña simultáneamente los papeles de patrocinador, gerente de proyecto, analista de requisitos y desarrollador. No existe un equipo, no existe un comité de control de cambios y no existe una contraparte que apruebe entregables desde fuera del proyecto.

Aun así, en el campo *quién* de cada proceso se nombra el rol que actúa en ese paso concreto. No es un formalismo vacío: separar los roles permite ver qué sombrero se lleva puesto en cada decisión y, sobre todo, permite ver dónde falta un contrapeso. La consecuencia de que el mismo individuo proponga, apruebe y ejecute los cambios de alcance se trata explícitamente en el apartado 4.2, porque afecta al control del alcance de una forma que conviene tener escrita antes que disimulada.

El cliente del sistema es la empresa BolivarBioingeniería LTDA. Su participación consta en la elicitación de requisitos que registra el apartado 1.2.4 de la ERS, pero no interviene en los ciclos de verificación descritos en el apartado 3 de este plan.

Los artefactos que sostienen la gestión del alcance

En el repositorio no hay rastro de herramienta externa de gestión de proyectos: ni tablero, ni sistema de tiquetes, ni integración continua configurada. El repositorio mismo cumple esa función, repartida entre los siguientes artefactos.

Artefacto	Dónde vive	Papel en la gestión del alcance
ERS IEEE 830	Documentation/IEEE830/	Enunciado del alcance del producto: 31 requisitos funcionales en nueve dominios, requisitos no funcionales y exclusiones explícitas
Checklist de reutilización	SecondBrain/wiki/malphasos/	Línea base del alcance del proyecto y su avance: 29 entradas, 24 cerradas y 5 pendientes, fechadas
Registro de decisiones técnicas	SecondBrain/wiki/malphasos/	Por qué el alcance es el que es: 17 apartados de decisiones con su justificación
Registro de deuda y riesgos	SecondBrain/wiki/patrones-reutilizables/	Defectos conocidos y trabajo diferido, separando lo heredado del sistema antecesor de lo propio
Bitácora	SecondBrain/log.md	Registro cronológico de solo adición: 15 entradas, una por avance o hallazgo
Migraciones Flyway	malphasos/src/main/resources/db/migration/	Línea base incremental e inmutable del modelo de datos: 18 tablas en cinco migraciones
Batería de pruebas	malphasos/src/test/	Criterio de aceptación objetivo: 339 pruebas en 36 clases
Historial de <i>git</i>	el repositorio completo	Registro de ejecución: 111 commits y 52 paquetes de trabajo integrados

Table 1: Artefactos del repositorio y la función de gestión del alcance que cumple cada uno.

Tres términos aparecen repetidamente y conviene fijarlos, porque el lector puede no conocer las herramientas:

- **git** es el sistema de control de versiones del proyecto. Registra cada cambio con su autor, su fecha y su justificación, y permite trabajar en *ramas*: líneas de desarrollo paralelas que luego se integran. En MalphasOS la rama es la unidad de paquete de trabajo, según se explica en el apartado 2.
- **Flyway** es la herramienta que aplica las migraciones de base de datos: archivos SQL numerados (V1, V2, ...) que se ejecutan una sola vez y en orden. Una vez aplicada, una migración no se edita; el cambio siguiente exige una migración nueva. Eso convierte la secuencia en un registro inmutable del alcance del modelo de datos.
- **Testcontainers** es la biblioteca que levanta un PostgreSQL real dentro de un contenedor para ejecutar las pruebas. Importa para la verificación del alcance porque las pruebas no corren contra una base de datos simulada, sino contra el mismo motor que usará producción.

1. Proceso de definición del alcance

1.1. Qué

Elaborar y mantener el enunciado del alcance: la declaración de qué construye MalphasOS y, con igual precisión, qué no construye.

En MSO ese enunciado no es un documento único, sino dos artefactos complementarios que responden a preguntas distintas:

- **El alcance del producto** lo enuncia la ERS (`Documentation/IEEE830/IEEE830.tex`). Contiene los 31 requisitos funcionales repartidos en nueve dominios —órdenes de trabajo, gestión de clientes, reportes de mantenimiento, firma digital, hojas de vida de los equipos, inventario, alertas y calibración, módulo comercial, y usuarios y seguridad—, los requisitos no funcionales y, en su apartado 1.2.2, las exclusiones declaradas: sin integración con sistemas externos de terceros, sin gestión contable o financiera, sin facturación, sin vigilancia de calibraciones mediante alertas automáticas y sin inventario de herramientas ni de equipos patrón, quedando las cotizaciones sujetas a priorización.
- **El alcance del proyecto** lo enuncia el wiki técnico, en el directorio `SecondBrain/wiki/malphasos/`: las notas `alcance-malphasos.md` y `checklist-reutilizacion.md`. MalphasOS se extrae de un sistema antecesor, `bolivarbioingenieria-app`, y una parte sustancial de la definición del alcance consistió en decidir, pieza por pieza, qué se porta tal cual, qué se reconstruye y qué se descarta.

La distinción no es académica. La ERS dice qué debe hacer el sistema terminado; el wiki dice qué trabajo entra en este proyecto. Hay cosas que están en el alcance del producto y todavía no en el del proyecto —el frontend completo, por ejemplo— y cosas del sistema antecesor que se leyeron y se decidió no traer.

1.2. Quién

- El **patrocinador** fija los límites exteriores: qué queda fuera del proyecto y con qué prioridad entra lo demás.
- El **analista de requisitos** redacta y mantiene la ERS, y traduce la lectura del sistema antecesor a las notas de alcance del wiki.
- El **gerente de proyecto** aprueba el enunciado resultante y decide cuándo se revisa.

Los tres roles los ejerce la misma persona, según lo advertido en la nota preliminar.

1.3. Cómo

La definición del alcance se hizo en tres movimientos, y se mantiene con una regla.

Primero, la elicitación con el cliente. Según registra el apartado 1.2.4 de la ERS, el alcance del producto se levantó mediante entrevistas, encuestas y reuniones con la gerente y un ingeniero de BolivarBioingeniería LTDA, más el análisis de los formatos que la empresa usa hoy para las órdenes de trabajo, los mantenimientos preventivos y las hojas de vida de los equipos. De ahí salieron el listado de requerimientos, el listado de requisitos y la matriz de trazabilidad con los que la ERS se declara consistente.

Segundo, la lectura del sistema antecesor. Antes de escribir una línea de código se exploró `bolivarbioingenieria-app` por completo y se volcó el resultado en un wiki de 43 notas enlazadas entre sí. Cada nota lleva una etiqueta de reutilización explícita —`reusable:alta`, `media`, `baja` o `no`— que es un juicio del proyecto, no un dato del sistema leído. Esa etiqueta es el instrumento con el que se recortó el alcance: lo marcado como `reusable:no` queda fuera por decisión razonada y con la razón escrita, no por olvido. El repositorio antecesor es de solo lectura y nunca se modifica desde este proyecto.

Tercero, el registro de cada decisión que mueve el alcance. Toda decisión que estreche o ensanche el enunciado se anota en la nota `decisiones-tecnicas-malphasos.md` del wiki, en el espíritu de un registro de decisiones de arquitectura ligero: qué se eligió y por qué. A la fecha reúne 17 apartados. Dos ejemplos de recorte deliberado: el despachador de eventos por RabbitMQ no se porta todavía —y cuando se porte hay que corregir antes un defecto conocido de encaminamiento—, y el adaptador de reportes de países se deja fuera por pertenecer a un módulo de reportes que este proyecto no aborda.

Y la regla que gobierna el mantenimiento: nada entra al enunciado sin evidencia, y toda afirmación que resulte falsa se corrige dejando constancia de que se corrigió y cuándo. Hay precedentes verificables de esa corrección. El wiki afirmaba que los dos archivos de exportación del realm de identidad eran idénticos, y no lo son; la afirmación se rectificó con fecha en lugar de sustituirse en silencio.

1.4. Cuándo

La definición inicial del alcance del producto se cerró con la primera versión de la ERS, fechada el 11 de agosto de 2026. La definición del alcance del proyecto se hizo entre el 27 de agosto de 2026 —fecha del primer commit del repositorio y de la carga inicial del wiki— y los días inmediatamente siguientes.

A partir de ahí, el enunciado se revisa en tres momentos:

- **Antes de abrir un módulo nuevo.** Se relee lo que el wiki dice de esa pieza en el sistema antecesor y se decide qué se trae, antes de escribir código y no después.
- **Cuando un hallazgo desmiente el enunciado.** Ocurre con frecuencia: al migrar un módulo aparecen defectos y supuestos falsos que obligan a rectificar la nota correspondiente en la misma sesión.
- **Cuando hay que pronunciarse sobre el estado del proyecto.** La revisión del 2 de septiembre de 2026 recorrió los 31 requisitos funcionales uno a uno y marcó cada uno contra el código. Se describe en el apartado 3.

1.5. Dónde

Todo el alcance vive en un único repositorio, `MalphasOS/`, con tres directorios de propósito separado:

- `Documentation/` — el enunciado formal del alcance del producto y sus diagramas. Es el entregable que se lee sin conocer el código.
- `SecondBrain/` — el wiki técnico: alcance del proyecto, decisiones, deuda y bitácora.
- `malphasos/` — el sistema construido, que es la única fuente de verdad sobre qué se ha entregado realmente.

El repositorio antecesor **bolivarbioingenieria-app** queda fuera de este árbol y se trata como fuente inmutable.

1.6. Con qué

La ERS se escribe en LaTeX y se compila a PDF con `latexmk`; el PDF se versiona para que el documento pueda leerse sin tener LaTeX instalado, mientras que los archivos intermedios de compilación quedan excluidos del control de versiones. Los diagramas de casos de uso se generan con PlantUML.

El wiki se escribe en Markdown con encabezado estructurado y enlaces internos entre notas, de modo que el mapa de decisiones sea navegable.

Una salvedad que conviene declarar aquí: los archivos fuente `.puml` de los diez diagramas de casos de uso de la ERS **no están en el repositorio** —solo están las imágenes ya compiladas—, de modo que los diagramas no pueden regenerarse ni corregirse. Varios de ellos describen dominios que el sistema no implementa. Es una limitación real del instrumental de definición del alcance y está registrada como deuda.

2. Proceso de elaboración de la EDT/WBS

2.1. Qué

Descomponer el alcance enunciado en paquetes de trabajo suficientemente pequeños para poder estimarse, ejecutarse y darse por cerrados de uno en uno; aprobarlos; y mantener la descomposición al día conforme el proyecto avanza.

MSO no dibuja la EDT como un árbol en un documento aparte. La descomposición existe, es explícita y está viva, pero materializada en dos niveles:

- **Nivel de planificación: el checklist de reutilización** (`SecondBrain/wiki/malphasos/checklist-reutilizacion.md`). Organiza el trabajo en siete grupos numerados —(1) infraestructura base, (2) esqueleto de aplicación, (3) primer módulo de dominio, (4) módulo de mantenimiento preventivo, (5) módulo de clientes, (6) identidad y frontend, (7) opcionales—, con 29 entradas marcables. Es la línea base del alcance y, a la vez, el indicador de avance: 24 entradas cerradas y 5 pendientes al 2 de septiembre de 2026.
- **Nivel de ejecución: una rama de *git* por paquete de trabajo**. Cada entrada del checklist se descompone en ramas nombradas por el alcance que entregan: `feat/equipment-schema`, `feat/client-domain`, `fix/client-error-codes`, `docs/wiki-equipment-completo`. El historial acumula 52 paquetes ya integrados.

El equivalente funcional del diccionario de la EDT —la descripción de qué contiene cada paquete y qué lo da por terminado— son el cuerpo del mensaje de commit, que explica en prosa por qué se hizo el cambio, y las notas de hallazgos por módulo del wiki. **MSO no mantiene un diccionario de la EDT en el sentido formal del término**, y este plan no va a fingir que sí.

2.2. Quién

- El **gerente de proyecto** descompone el alcance en grupos, los ordena por dependencia y decide el tamaño de cada paquete.
- El **analista de requisitos** comprueba que cada paquete corresponda a algo enunciado en el alcance, y no a trabajo que se coló sin estar pedido.
- El **desarrollador** ejecuta el paquete y lo entrega para revisión.
- La **aprobación** de cada paquete es una decisión del gerente de proyecto y se materializa en la revisión previa al commit descrita en el apartado 3.

2.3. Cómo

Del enunciado a los grupos, por dependencia técnica y no por valor percibido. El checklist está ordenado de modo que ningún grupo empiece antes de que exista aquello sobre lo que se apoya: primero la infraestructura —base de datos, proveedor de identidad, contenedores—, después el esqueleto de la aplicación —eventos de dominio, seguridad, manejo de errores, documentación de la API—, y solo entonces los módulos de negocio. El primer grupo lleva escrita su propia razón de ser primero: sin él no hay nada que construir encima.

De un grupo a paquetes, cortando por agregado antes que por capa. Ésta es la regla de descomposición del proyecto y está escrita como tal. Un paquete de trabajo entrega una unidad de negocio completa a través de todas sus capas, en lugar de entregar una capa horizontal de todo el módulo. La razón es de verificabilidad: una capa suelta no se puede probar, mientras que un agregado completo sí.

Dentro de un módulo, la secuencia observable es esquema $\rightarrow$ dominio $\rightarrow$ aplicación y persistencia $\rightarrow$ exposición REST. El módulo de equipos lo ilustra exactamente: se descompuso en cinco paquetes, `feat/equipment-schema`, `feat/equipment-domain`, `feat/equipment-catalog-base`, `feat/equipment-chain` y `feat/equipment-rest`, que corresponden a esas cuatro etapas, con la tercera partida en dos por tamaño.

El criterio que fija el tamaño mínimo del paquete es objetivo: un paquete es el trozo más pequeño que deja el sistema funcionando y la batería de pruebas en verde. La regla se enuncia así en las convenciones del proyecto: si separar dos piezas deja las pruebas rotas —un servicio sin su adaptador impide arrancar la aplicación—, esas dos piezas van juntas en el mismo paquete. No hay margen de interpretación: lo decide la ejecución de las pruebas, no una estimación.

Aprobación. Ningún paquete se cierra sin que el responsable haya revisado la diferencia completa que introduce. Esa revisión es el punto de control real del proyecto y se detalla en el apartado 3.3.

Mantenimiento de la descomposición. Al cerrar un paquete se marca su casilla en el checklist con la fecha de cierre, se añade una entrada a la bitácora y, si el paquete produjo hallazgos, se actualizan las notas afectadas del wiki. Es una regla permanente del proyecto: el wiki se mantiene en la misma sesión de trabajo y sin que nadie lo pida. La EDT, por tanto, no se rehace periódicamente en una reunión de replanificación; se edita cada vez que un paquete se cierra.

Trazabilidad del paquete. Los paquetes se integran a la línea principal con un merge que conserva la bifurcación en el historial, y la rama se borra tras integrarse. El resultado es que cada paquete deja en el historial un commit de integración que enuncia qué entregó, y el detalle de cómo se hizo queda debajo. Además, los trabajos de documentación se separan de los de producto: se hacen en ramas con prefijo `docs/`, tocan únicamente `Documentation/` o `SecondBrain/`, y no se integran sin revisión explícita del responsable.

2.4. Cuándo

La descomposición de un grupo se elabora justo antes de abordarlo, no al inicio del proyecto. El checklist se creó completo en su primer nivel el 27 de agosto de 2026, pero la partición fina en paquetes de cada módulo se decidió al abrirlo, con la información que las migraciones anteriores ya habían producido.

La actualización es continua y ligada al cierre de cada paquete, no calendarizada. El ritmo observable: 111 commits y 52 paquetes integrados entre el 27 de agosto y el 2 de septiembre de 2026.

2.5. Dónde

El nivel de planificación, en `SecondBrain/wiki/malphasos/checklist-reutilizacion.md`. El nivel de ejecución, en el grafo de ramas del repositorio, consultable con el historial de *git*. La bitácora de cierres, en `SecondBrain/log.md`.

2.6. Con qué

Git aporta las ramas —la unidad de paquete—, el merge que conserva la bifurcación y el historial que permite reconstruir qué entregó cada uno. Markdown sostiene el checklist con casillas marcables, que es la forma más barata que tiene el proyecto de mostrar avance sin mantener un sistema aparte. Maven, con la batería de pruebas y Testcontainers, aporta el criterio que decide si un paquete es lo bastante grande para cerrarse.

3. Proceso para la verificación del alcance

3.1. Qué

Comprobar que lo construido es lo que el enunciado prometía y dejar constancia de su aceptación. En MSO la verificación opera en dos niveles distintos, con instrumentos distintos:

- **Verificación por paquete**, continua: se ejerce sobre cada paquete de trabajo antes de darlo por cerrado.
- **Verificación del alcance total**, periódica: el contraste requisito por requisito de la ERS contra el código, que dice qué parte del alcance enunciado está realmente entregada.

3.2. Quién

- El **desarrollador** ejecuta la batería de pruebas y verifica manualmente el comportamiento contra la aplicación en ejecución.
- El **gerente de proyecto** revisa la diferencia completa antes del commit y otorga —o niega— el visto bueno. **Éste es el punto de aceptación formal del proyecto.**
- El **analista de requisitos** realiza el contraste entre la ERS y el código, y marca el estado de cada requisito.

No hay aceptación por parte de un cliente externo. No existe acta de aceptación firmada por BolivarBioingeniería LTDA, ni un ciclo de entrega y revisión con la empresa. La aceptación la otorga el mismo responsable que produjo el entregable, y su único registro es el commit. Es una limitación del proyecto, no una simplificación de este documento.

3.3. Cómo

La verificación se apoya en cuatro mecanismos, ordenados del más frecuente al más excepcional.

Primero, la batería de pruebas como criterio objetivo de aceptación. Al 2 de septiembre de 2026 son **339 pruebas repartidas en 36 clases, con cero fallos, cero errores y cero omitidas**, verificado ejecutando la batería completa. Lo que la hace un criterio de aceptación y no un mero indicador de salud es dónde corre: Testcontainers levanta un PostgreSQL 17 real, el mismo motor que usará producción, de modo que las migraciones se aplican de verdad y las restricciones del esquema se comprueban de verdad. Una base de datos en memoria habría obligado a escribir SQL al mínimo común denominador y habría dejado sin verificar precisamente las reglas de negocio que el esquema expresa.

La regla asociada es tajante: **cada commit queda con la batería en verde.** Un paquete que la deja en rojo no se cierra; se completa o se junta con el que le falta.

La batería no verifica solo que el sistema arranque. Verifica comportamiento: que las migraciones corrieron, que una petición sin credenciales recibe 401 y una con credenciales insuficientes recibe 403, que cada recurso de la API aparece efectivamente en la documentación que el sistema publica. Este último caso ilustra bien el valor del mecanismo:

la ausencia de un recurso en la documentación publicada es un fallo silencioso —no hay error de arranque, no hay traza en el registro, el recurso simplemente no aparece— y se produjo cuatro veces antes de que una prueba lo fijara.

Segundo, la revisión de la diferencia antes de cada commit. Ningún cambio se integra sin que el responsable haya leído línea por línea qué introduce. Es el control de aceptación real del proyecto y el único que examina la intención además del resultado: las pruebas dicen que el sistema funciona, la revisión dice si hace lo que se quería y solo eso. La regla complementaria —una preocupación por commit— existe justamente para que la revisión sea posible: un cambio que mezcla dos asuntos no se puede aceptar ni rechazar como una unidad.

Una regla más, verificable en la forma de trabajar: **se verifica ejecutando, no compilando**. Varios de los defectos encontrados en el sistema antecesor compilaban perfectamente.

Tercero, las migraciones numeradas como línea base verificable del modelo de datos. El esquema de la base de datos no lo genera el código: lo definen cinco archivos SQL numerados que Flyway aplica en orden y una sola vez. `V1__baseline.sql` no crea tablas, solo establece el punto de partida del versionado; `V2__person.sql` aporta tres tablas, `V3__location.sql` dos, `V4__client.sql` siete y `V5__equipment_catalog.sql` seis, para un total de 18. Una migración aplicada no se edita nunca; el cambio siguiente exige una migración nueva.

La pieza que convierte esto en verificación y no en mera documentación es la configuración `ddl-auto: validate`: la capa de persistencia tiene prohibido modificar el esquema y se limita a comprobar que lo que el código declara coincide con lo que las migraciones crearon. Si divergen, la aplicación no arranca. El alcance del modelo de datos queda así fijado por un artefacto inmutable y comprobado automáticamente en cada ejecución.

Cuarto, el contraste de la ERS contra el código. Éste es el mecanismo de verificación del alcance total del proyecto, y merece decirse con ese nombre: es lo único que se pronuncia sobre el enunciado mismo y dice qué parte está entregada y qué parte no.

El 2 de septiembre de 2026 se recorrieron los 31 requisitos funcionales del apartado 3.2 de la ERS, uno a uno, contrastándolos con los controladores REST, los puertos de entrada de cada módulo, las reglas del dominio y las cinco migraciones. Cada requisito lleva ahora una de dos marcas y un punto de evidencia:

- **[IMPLEMENTADO]** — hay código que hace lo que el requisito pide, y el punto de evidencia cita la ruta, el agregado o la tabla concreta.
- **[PREVISTO]** — el requisito sigue formando parte del alcance, pero no hay una línea de código que lo implemente, y el punto de evidencia dice qué se comprobó para afirmarlo.

El resultado: 8 requisitos implementados y 23 previstos. Lo entregado son los datos maestros —personas con su alta en el proveedor de identidad, países y ciudades, clientes con sus sedes y áreas de servicio, y el catálogo e inventario de equipos—. Lo que falta es el flujo operativo que se apoya sobre ellos: órdenes de trabajo, reportes de mantenimiento, firma digital, inventario de existencias, alertas de calibración y cotizaciones.

Tres precisiones importan, porque son casos en los que la marca de implementado no significa cumplimiento completo y la salvedad se enuncia junto a la marca en lugar de esconderse tras ella:

- El criterio de RF-50 de que cada rol acceda a lo suyo **no se cumple**: las 83 operaciones de la API exigen la misma autoridad de administrador y los grupos de ingenieros y de clientes no la reciben.
- Los cambios sobre un usuario del RF-52 no se propagan al proveedor de identidad.
- La baja de un usuario del RF-53 no revoca su acceso, porque no toca su cuenta en el proveedor de identidad.

Un efecto lateral del contraste merece constar, porque cambia cómo se entiende esta actividad: **comparar la especificación con su implementación resultó ser una técnica de detección de defectos, no solo de documentación**. El ejercicio del 2 de septiembre encontró seis defectos de código —dos de ellos de seguridad y de severidad alta— que ninguna lectura del módulo donde vivían habría revelado. Aparecen únicamente al preguntarle a un documento qué promete y al código qué cumple.

3.4. Cuándo

La batería de pruebas y la revisión de la diferencia se ejecutan **antes de cada commit**, sin excepción. La comprobación manual extremo a extremo contra la aplicación en contenedores se hace al cerrar un módulo.

El contraste de la ERS contra el código se realiza cuando hay que pronunciarse sobre el estado del alcance. Se hizo por primera vez el 2 de septiembre de 2026, con los cuatro módulos de dominio cerrados. No tiene periodicidad fijada; el momento natural para repetirlo es el cierre del módulo siguiente.

3.5. Dónde

Las pruebas, en `malphasos/src/test/`. Las migraciones, en `malphasos/src/main/resources/db/migration/`. El resultado del contraste, en el apartado 3.2 de la ERS, marcado requisito por requisito. La constancia de cada aceptación, en el historial de *git*. Los hallazgos derivados de la verificación, en el registro de deuda del wiki.

3.6. Con qué

Maven ejecuta la batería; JUnit 5 la estructura; Testcontainers levanta el PostgreSQL 17 sobre el que corre; Flyway aplica y valida las migraciones. Docker Compose levanta el entorno completo —base de datos, proveedor de identidad y mensajería— para la verificación manual extremo a extremo. La documentación de la API que el sistema publica sirve de instrumento para comprobar que la superficie expuesta es la declarada. Y `git diff` es el instrumento de la revisión previa al commit.

Una limitación conocida del instrumental: la batería ha sido intermitente. La comprobación de salud del servicio de mensajería intenta conectarse a un puerto local y falla si el contenedor no está levantado, lo que en su día produjo dos resultados distintos para el mismo código. El 2 de septiembre de 2026 varias ejecuciones completas salieron limpias, pero que no se reproduzca no cierra el problema: la comprobación sigue apuntando al mismo puerto. Una batería que puede dar dos resultados distintos deja de servir como criterio de aceptación, y por eso la corrección está registrada como pendiente.

4. Proceso para el control del alcance

4.1. Qué

Identificar los cambios de alcance, registrarlos, decidir si entran y mantener la línea base coherente con lo decidido.

En MSO los cambios de alcance no llegan como solicitudes de un cliente. Llegan por cuatro vías observables:

- **Del contraste entre documentos y código:** la revisión del apartado 3 produjo seis defectos y cinco observaciones sobre la propia ERS.
- **De la comparación de módulos entre sí:** es el patrón que más defectos ha producido en este proyecto. Los fallos de las capas externas no aparecen leyendo un módulo, sino preguntándose si el fallo recién corregido en uno tiene más instancias en los otros.
- **De la deuda heredada:** el sistema antecesor acumula defectos conocidos que obligan a decidir, pieza por pieza, si se corrigen al portar o se dejan fuera.
- **De decisiones técnicas que estrechan el alcance deliberadamente,** como aplazar el despachador de eventos por mensajería o dejar fuera el módulo de reportes.

4.2. Quién

- Quien **detecta** el cambio es el desarrollador —cuando aparece al construir— o el analista de requisitos —cuando aparece al contrastar.
- Quien **decide** si entra, se aplaza o se rechaza es el gerente de proyecto.
- Quien **asume el costo** de la decisión es el patrocinador.

Y aquí la concentración de roles deja de ser un detalle administrativo. Los tres son la misma persona: quien propone el cambio, quien lo aprueba y quien paga sus consecuencias. **No hay contraparte independiente que pueda rechazar un cambio de alcance.** El control carece del contrapeso que en un proyecto con comité proporciona la separación de intereses, y los dos riesgos concretos son la fluencia del alcance por decisión propia —ampliar lo que se está construyendo porque resulta interesante— y, sobre todo, la aceptación complaciente: nadie más va a mirar un entregable y decir que no cumple.

Cuatro mecanismos lo sustituyen parcialmente, y conviene ser preciso sobre qué cubre cada uno:

- **La batería de pruebas es el único juez que no negocia.** Dice verde o rojo con independencia de quién la ejecute y de lo que esa persona desee. Es el sustituto más sólido, y es el motivo de que la intermitencia señalada en el apartado 3.6 sea un problema de control de alcance y no solo una molestia técnica.
- **El registro escrito y fechado obliga a justificarse.** Una decisión que hay que escribir en el registro de decisiones, con su porqué y su fecha, queda expuesta a la lectura futura. No es una aprobación externa, pero encarece la decisión arbitraria.

- **El contraste ERS↔código enfrenta dos artefactos, no dos opiniones.** Un requisito no se puede dar por implementado porque a alguien se lo parezca: hay que citar la ruta, el agregado o la tabla, o admitir que no existe. Es la aproximación más cercana a una revisión independiente que el proyecto tiene, y funciona porque el artefacto contrastado no puede ser persuadido.
- **La espera deliberada entre producir y aceptar.** Los trabajos de documentación se dejan en su rama y no se integran a la línea principal hasta que el responsable los revisa por separado. Introduce una separación en el tiempo entre escribir y aceptar, que es lo poco que puede hacerse cuando no hay separación entre personas.

Lo que ninguno de los cuatro sustituye: los cuatro comprueban consistencia interna —que el sistema hace lo que el proyecto dice que hace—. Ninguno comprueba que lo construido sea lo que el cliente necesita. La validación con BolivarBioingeniería LTDA está fuera de lo que el repositorio evidencia, y este plan no puede afirmar que ocurra.

4.3. Cómo

Identificación. Un cambio de alcance se manifiesta de tres formas reconocibles: una entrada nueva en el checklist, una entrada nueva en el registro de deuda, o un requisito de la ERS que cambia de marca.

Registro. MSO mantiene cuatro registros con propósitos deliberadamente distintos, y no conviene fundirlos:

Registro	Qué guarda y por qué está separado
Checklist de reutilización	La línea base viva y su avance. Responde "¿qué falta?". 24 entradas cerradas y 5 abiertas
Decisiones técnicas	Qué se decidió y por qué. Responde "¿por qué el alcance es éste?". 17 apartados
Deuda técnica y riesgos	Defectos conocidos y trabajo diferido, con la deuda heredada separada de la propia. Responde "¿qué sabemos que está mal y decidimos no arreglar todavía?"
Bitácora	Registro cronológico de solo adición, 15 entradas. Responde "¿qué pasó y cuándo?"

Table 2: Los cuatro registros del control del alcance y la pregunta que responde cada uno.

Una convención del registro de deuda merece destacarse porque protege la trazabilidad: **las entradas cerradas no se borran.** Se tachan y se fecha el cierre, con una nota de cómo se resolvió. Así queda constancia de lo que en su momento se decidió no hacer y luego se hizo, que es exactamente el rastro que un control de cambios necesita conservar.

Procesamiento. Un cambio aceptado se convierte en un paquete de trabajo, con su rama; un cambio aplazado queda registrado como pendiente con su razón y su severidad. **La distinción se respeta incluso cuando corregir sería más rápido que anotar.** Los seis defectos que produjo el contraste del apartado 3 —dos de ellos de seguridad y de severidad alta— se anotaron como pendientes en vez de corregirse sobre la marcha, porque el paquete en curso era de documentación y corregir código dentro de él habría mezclado dos alcances en un mismo entregable. Es la aplicación estricta de la regla de una preocupación por paquete.

El alcance se controla marcando, no recortando. La revisión de la ERS no eliminó ninguno de los 23 requisitos sin código detrás: siguen siendo el alcance del proyecto y lo único que cambió es que dejaron de leerse como cosas que el sistema ya hace. Eliminarlos habría reducido el alcance por la vía de fingir que nunca se prometió; marcarlos conserva el compromiso y hace visible la distancia que falta.

Corrección de la línea base. Cuando una afirmación del enunciado resulta falsa, se corrige dejando constancia de la corrección y su fecha, en lugar de sustituirla en silencio. Hay precedentes verificables además del ya citado: se afirmó que un módulo daba código de error propio a cada referencia externa, se comprobó y era falso; la comprobación produjo la corrección y la rectificación de la nota.

Enlace con el control integrado de cambios. MSO ejerce el control integrado en un único punto: **la revisión de la diferencia previa al commit**. Ahí convergen las tres dimensiones que un control integrado debe conciliar:

- **Alcance** — la revisión comprueba que el cambio entrega lo que el paquete prometía y nada más.
- **Calidad** — el cambio no entra si deja la batería en rojo, y esa condición no es negociable en la revisión.
- **Cronograma** — un cambio que no cabe en un commit revisable se parte en dos paquetes, lo que altera la descomposición de la EDT en el mismo acto.

La separación por ramas es la que impide que un cambio de alcance se cuele dentro de otro: una preocupación por rama, una por commit. Y la separación adicional entre las ramas que tocan el producto y las que tocan la documentación evita el caso inverso, que un cambio de documento arrastre un cambio de producto sin que nadie lo advierta.

4.4. Cuándo

El control del alcance es continuo, no calendarizado:

- La **identificación** ocurre en cualquier momento, y con más frecuencia al comparar dos artefactos que al leer uno.
- El **registro** se hace en la misma sesión de trabajo del hallazgo. Es una regla explícita del proyecto: ante un cambio grande o un hallazgo relevante, el wiki se actualiza sin que nadie lo pida, incluidos su índice y su bitácora.
- El **procesamiento** ocurre en el siguiente ciclo de rama, o queda diferido con su razón escrita.
- El **control integrado** se ejerce en cada commit, que es la frecuencia máxima posible.

4.5. Dónde

Los cuatro registros, en **SecondBrain/**: el checklist y las decisiones en **wiki/malphasos/**, la deuda en **wiki/patrones-reutilizables/**, y la bitácora en la raíz del wiki. El estado por requisito, en el apartado 3.2 de la ERS. El rastro de ejecución de cada cambio, en el historial de *git*.

4.6. Con qué

Git sostiene la separación por ramas que impide mezclar cambios, y su historial es el registro de qué se aceptó y cuándo. Markdown sostiene los cuatro registros, con una convención de tachado y fecha que preserva lo cerrado. Maven, JUnit y Testcontainers aportan el criterio que ningún rol puede negociar. Y `git diff` es el instrumento del único punto de control integrado que el proyecto tiene.

Todas las cifras de este plan corresponden al estado del proyecto al 2 de septiembre de 2026 y se comprobaron sobre el repositorio: 111 commits, 52 paquetes de trabajo integrados, cuatro módulos de dominio cerrados, 18 tablas en cinco migraciones, 83 operaciones REST expuestas, 339 pruebas sin fallos, y 8 de 31 requisitos funcionales implementados.